Employee ID Management Using BST

C++ Program — Insertion, Searching, Deletion, and Inorder Display

```
#include <iostream>
using namespace std;

struct Node {
    int employeeID;
    Node* left;
    Node* right;
};

Node* createNode(int id) {
    Node* newNode = new Node;
    newNode->employeeID = id;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Insert employee ID
Node* insert(Node* root, int id) {
    if (root == NULL)
        return createNode(id);

    if (id < root->employeeID)
        root->left = insert(root->left, id);
    else if (id > root->employeeID)
        root->right = insert(root->right, id);
    else
        cout << "Employee ID already exists!\n";

    return root;
}

// Search employee ID
bool search(Node* root, int id) {
```

```
    if (root == NULL)
        return false;

    if (root->employeeID == id)
        return true;

    if (id < root->employeeID)
        return search(root->left, id);

    return search(root->right, id);
}

// Find minimum node
Node* findMin(Node* root) {
    while (root != NULL && root->left != NULL)
        root = root->left;

    return root;
}

// Delete employee ID
Node* deleteNode(Node* root, int id) {
    if (root == NULL)
        return root;

    if (id < root->employeeID) {
        root->left = deleteNode(root->left, id);
    }
    else if (id > root->employeeID) {
        root->right = deleteNode(root->right, id);
    }
    else {
        // No left child
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
```

```
        }

        // No right child
        if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Two children
        Node* temp = findMin(root->right);
        root->employeeID = temp->employeeID;
        root->right = deleteNode(
            root->right, temp->employeeID
        );
    }

    return root;
}

// Display IDs in ascending order
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->employeeID << " ";
        inorder(root->right);
    }
}

// Free allocated memory
void freeTree(Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        delete root;
    }
}
```

```
int main() {
    Node* root = NULL;
    int choice, id;

    do {
        cout << "\n--- Employee ID Management ---\n";
        cout << "1. Insert Employee ID\n";
        cout << "2. Search Employee ID\n";
        cout << "3. Delete Employee ID\n";
        cout << "4. Display Employee IDs\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter Employee ID: ";
            cin >> id;
            root = insert(root, id);
            break;

        case 2:
            cout << "Enter Employee ID to search: ";
            cin >> id;

            if (search(root, id))
                cout << "Employee ID found!\n";
            else
                cout << "Employee ID not found!\n";
            break;

        case 3:
            cout << "Enter Employee ID to delete: ";
            cin >> id;

            if (search(root, id)) {
                root = deleteNode(root, id);
```

```
                cout << "Employee ID deleted successfully!\n";
            } else {
                cout << "Employee ID not found!\n";
            }
            break;

        case 4:
            cout << "Employee IDs in ascending order: ";
            inorder(root);
            cout << endl;
            break;

        case 5:
            cout << "Exiting program...\n";
            break;

        default:
            cout << "Invalid choice! Try again.\n";
        }

    } while (choice != 5);

    freeTree(root);
    return 0;
}
```